

Introduction To UML

- **Unified Modeling Language (UML)** is a general-purpose modeling language. The main aim of UML is to define a standard way to **visualize** the way a system has been designed.
- It is quite similar to blueprints used in other fields of engineering. UML is **not a programming language** , it is rather a visual language.
- Understanding and effectively using UML can significantly improve the quality and clarity of your software designs. Our specialized course on System design provides detailed guidance and practical examples to help you master this visual language.
- By integrating UML into your workflow, you can create more comprehensive and communicative system models.

Goals of UML

- Since it is a general-purpose modeling language, it can be utilized by all the modelers.
- UML came into existence after the introduction of object-oriented concepts to systemize and consolidate the object-oriented development, due to the absence of standard methods at that time.
- The UML diagrams are made for business users, developers, ordinary people, or anyone who is looking forward to understand the system, such that the system can be software or non-software.
- Thus it can be concluded that the UML is a simple modeling approach that is used to model all the practical systems.

Characteristics of UML

- It is a generalized modeling language.
- It is distinct from other programming languages like C++, Python, etc.
- It is interrelated to object-oriented analysis and design.
- It is used to visualize the workflow of the system.
- It is a pictorial language, used to generate powerful modeling artifacts.



Principles of Modeling

- Four basic principles of modeling
 1. The models we choose have a profound influence on the solution we provide
 2. Every model may be expressed at different levels of abstraction
 3. The best models are connected to reality
 4. No single model is sufficient, a set of models is needed to solve any nontrivial system



1. Principle

1. The models we choose have a profound influence on the solution we provide

- In other words, **choose your models well.**
- The right models will **brilliantly illuminate the most wicked development problems, offering insight** that you simply could not gain otherwise; the **wrong models will mislead you, causing** you to **focus on irrelevant issues.**



If you build a system through the eyes of a

- *database developer, you will likely focus on entity-relationship models that push behavior into triggers and stored procedures.*
- *structured analyst, you will likely end up with models that are algorithmic-centric, with data flowing from process to process.*
- *an object-oriented developer, you'll end up with a system whose architecture is centered around a sea of classes and the patterns of interaction that direct how those classes work together.*



2. Principle

2. *Every model may be expressed at different levels of abstraction*

- help your investors **visualize its look and feel.**
- **an analyst or an end user** will want to **focus on issues of what**
- **a developer** will want to **focus on issues of how**

3. Principle

3. *The best models are connected to reality*

- the **Achilles heel** (*a weakness or vulnerable point*) of structured analysis techniques is the fact that there is a **basic disconnect between its analysis model and the system's design model.**
- In **object-oriented systems**, it is **possible to connect all the nearly independent views of a system into one semantic whole.**



4. Principle

4. No single model is sufficient, a set of models is needed to solve any nontrivial system

- To understand the architecture of such a system, you need several complementary and interlocking views:
- a use case view (exposing the requirements of the system),
- a design view (capturing the vocabulary of the problem space and the solution space),

- 
- 
- an interaction view (showing the interactions among the parts of the system and between the system and the environment),
 - an implementation view (addressing the physical realization of the system), and
 - a deployment view (focusing on system engineering issues).

Each of these views may have structural, as well as behavioral, aspects.

Together, these views represent the blueprints of software.

Class Diagrams

- ❑ Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- ❑ Class diagram describes the attributes and operations of a class and also the constraints imposed on the system.
- ❑ The class diagrams are widely used in the modeling of object oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.
- ❑ Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Purpose of Class Diagrams

- ❑ The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.
- ❑ UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.

How to Draw a Class Diagram?

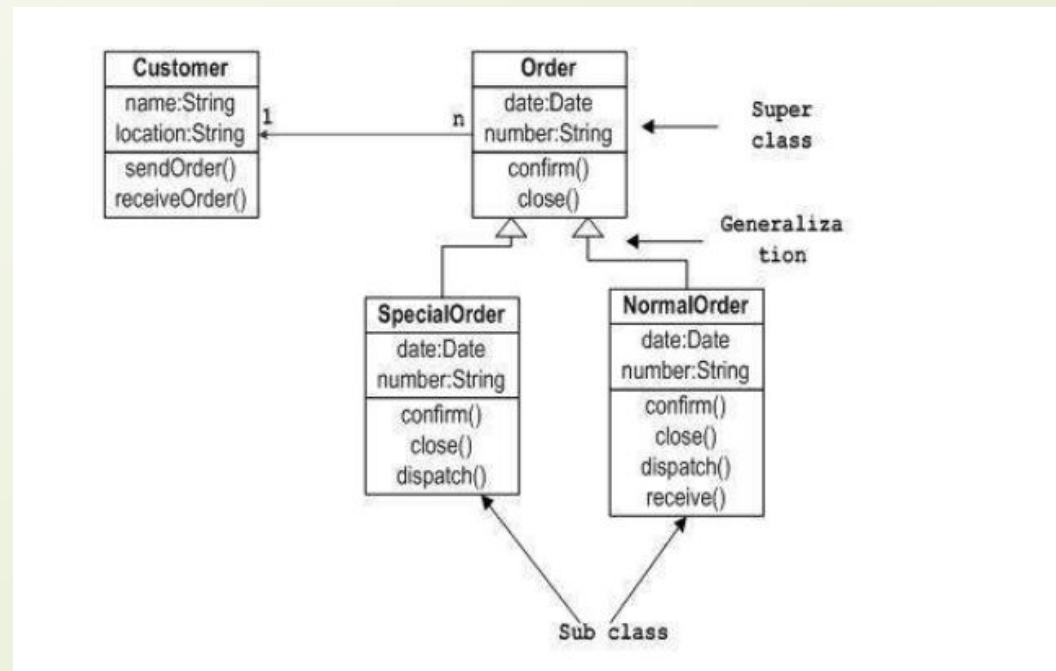
- ❑ Class diagrams are the most popular UML diagrams used for construction of software applications. It is very important to learn the drawing procedure of class diagram.
- ❑ Class diagrams have a lot of properties to consider while drawing but here the diagram will be considered from a top level view.
- ❑ Class diagram is basically a graphical representation of the static view of the system and represents different aspects of the application.

The following points should be remembered while drawing a class diagram

- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.
- Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

The following diagram is an example of an Order System of an application. It describes a particular aspect of the entire application.

- First of all, Order and Customer are identified as the two elements of the system. They have a one-to-many relationship because a customer can have multiple orders.
- Order class is an abstract class and it has two concrete classes (inheritance relationship) SpecialOrder and NormalOrder.
- The two inherited classes have all the properties as the Order class. In addition, they have additional functions like dispatch () and receive ().



Object Diagrams

- ❑ Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams.
- ❑ Object diagrams represent an instance of a class diagram. The basic concepts are similar for class diagrams and object diagrams. Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment.
- ❑ **Purpose of Object Diagrams**
- ❑ The purpose of a diagram should be understood clearly to implement it practically. The purposes of object diagrams are similar to class diagrams.
- ❑ The difference is that a class diagram represents an abstract model consisting of classes and their relationships. However, an object diagram represents an instance at a particular moment, which is concrete in nature.
- ❑ It means the object diagram is closer to the actual system behavior. The purpose is to capture the static view of a system at a particular moment.
- ❑ The purpose of the object diagram can be summarized as –
 - Forward and reverse engineering.
 - Object relationships of a system
 - Static view of an interaction.
 - Understand object behaviour and their relationship from practical perspective

How to Draw an Object Diagram

- It implies that an object diagram consists of instances of things used in a class diagram.
- So both diagrams are made of same basic elements but in different form. In class diagram elements are in abstract form to represent the blue print and in object diagram the elements are in concrete form to represent the real world object.
- To capture a particular system, numbers of class diagrams are limited.
 - First, analyze the system and decide which instances have important data and association.
 - Second, consider only those instances, which will cover the functionality.
 - Third, make some optimization as the number of instances are unlimited.
 - Object diagrams consist of objects.
 - The link in object diagram is used to connect objects.
 - Objects and links are the two elements used to construct an object diagram.

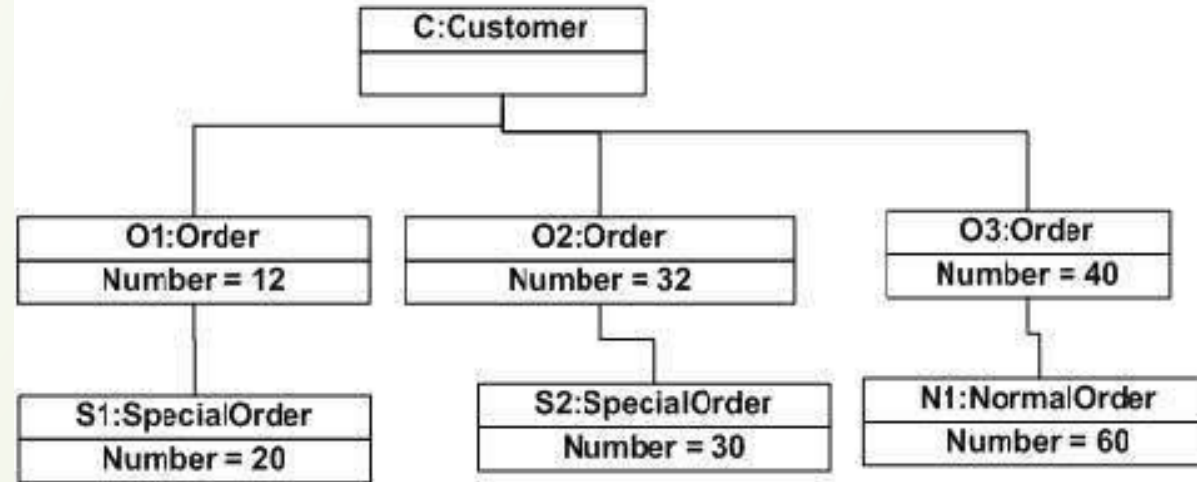
- 
- The following diagram is an example of an object diagram. It represents the Order management system which we have discussed in the chapter Class Diagram.

The following diagram is an instance of the system at a particular time of purchase. It has the following objects.

- Customer
- Order
- Special Order
- Normal Order

- Now the customer object (C) is associated with three order objects (O1, O2, and O3). These order objects are associated with special order and normal order objects (S1, S2, and N1). The customer has the following three orders with different numbers (12, 32 and 40) for the particular time considered.
- The customer can increase the number of orders in future and in that scenario the object diagram will reflect that. If order, special order, and normal order objects are observed then you will find that they have some values.
- For orders, the values are 12, 32, and 40 which implies that the objects have these values for a particular moment (here the particular time when the purchase is made is considered as the moment) when the instance is captured
- The same is true for special order and normal order objects which have number of orders as 20, 30, and 60. If a different time of purchase is considered, then these values will change accordingly.

Object diagram of an order management system



❑ Use Object Diagrams

- A particular state which is running.
- A particular number of passengers. which will change if the snap is taken in a different time
- Making the prototype of a system.
- Reverse engineering.
- Modeling complex data structures.
- Understanding the system from practical perspective.